

Relatório de Cobertura de Código & Análise de Qualidade

Avaliação crítica de QA — módulo `ServicoDePagamento`

Prova — Módulo 3 · PGATS · Sistema de Pagamento com Testes

Autor do código: Gabriel Guimarães Nunes | Avaliação por: QA Engineer
Data: 21/06/2026 | Versão: 2 (pós-implementação dos testes faltantes)
Ferramentas: Mocha 8 · c8 (Istanbul) · Node 22

1. Sumário executivo

O repositório implementa a classe `ServicoDePagamento` com dois métodos públicos (`pagar` e `consultarUltimoPagamento`). A suíte original de **9 testes** foi reestruturada e ampliada para **28 testes** — **23 ativos (verdes)** e **5 pendentes** (`it.skip`) que documentam defeitos. Todos seguem o padrão **AAA** e cobrem as técnicas de Partição de Equivalência, Valor-Limite, Tabela de Decisão e MC/DC. A medição de cobertura foi reexecutada com **c8 / Istanbul**.

Atualização v2: após a implementação, a suíte passou de 9 → 28 testes (23 passando, 5 pendentes). A cobertura estrutural **permaneceu em 100%** em todas as métricas, pois o código de produção não foi alterado (decisão de QA: documentar os bugs sem mascará-los). Os 5 testes pendentes ficam prontos como *testes de regressão* — basta remover o `.skip` quando o código for corrigido.

100%

STATEMENTS
27 / 27

100%

BRANCHES
10 / 10

100%

FUNCTIONS
3 / 3

100%

LINES
27 / 27

Resultado da cobertura estrutural: 100% em todas as métricas. A suíte exercita todas as linhas, ramos e funções do código de produção, e os 9 testes passam.

Mas cobertura ≠ ausência de defeitos. Aplicando partição de equivalência, análise de valor-limite e tabela de decisão, foram identificados **5 defeitos / lacunas de regra de negócio** que os 100% de cobertura *não* capturam — comprovados empiricamente na Seção 6. O número de cobertura é alto porque o código é pequeno, não porque a validação é robusta.

2. O que foi medido

A cobertura foi obtida rodando a suíte real sob instrumentação Istanbul:

```
# comando executado
npx c8 --reporter=text --reporter=json-summary mocha

# resultado (resumo Istanbul)
-----|-----|-----|-----|-----|
File      | % Stmts | % Branch | % Funcs | % Lines |
-----|-----|-----|-----|-----|
All files |    100 |    100 |    100 |    100 |
ServicoDePagamento.js |    100 |    100 |    100 |    100 |
```

-----|-----|-----|-----|-----|
9 passing

Métrica	Coberto	Total	%	Visual
Statements (instruções)	27	27	100%	
Branches (ramos de decisão)	10	10	100%	
Functions (funções)	3	3	100%	
Lines (linhas)	27	27	100%	

As 3 funções contabilizadas pelo Istanbul são: o inicializador de campos de instância (`#pagamentos = []`), `pagar` e `consultarUltimoPagamento` .

3. Código sob análise & regras de negócio

```
export class ServicoDePagamento {
  #pagamentos = [];

  pagar(codigoBarras, empresa, valor) {
    const categoria = valor > 100 ? "cara" : "padrão"; // D1 - categorização
    const pagamento = { codigoBarras, empresa, valor, categoria };
    if (valor <= 0) { // D2 - validação (depois!)
      throw new Error("Valor do pagamento deve ser maior que zero.");
    }
    this.#pagamentos.push(pagamento);
  }

  consultarUltimoPagamento() {
    if (this.#pagamentos.length === 0) { // D3 - lista vazia
      throw new Error("Nenhum pagamento registrado.");
    }
    return this.#pagamentos[this.#pagamentos.length - 1];
  }
}
```

Regras de negócio extraídas

#	Regra	Comportamento esperado
RN1	Valor ≤ 0	Rejeitar com erro "Valor do pagamento deve ser maior que zero."
RN2	Valor > 100	Categoria "cara"
RN3	0 < Valor ≤ 100	Categoria "padrão" (limite 100 é exclusivo)
RN4	Consulta sem pagamentos	Rejeitar com erro "Nenhum pagamento registrado."
RN5	Múltiplos pagamentos	Sempre retornar o último inserido

4. Técnicas de teste aplicadas

4.1 Partição de equivalência — parâmetro `valor`

Classe	Faixa / tipo	Saída esperada	Coberta pela suíte?
CE1 — inválida (nula/negativa)	<code>valor ≤ 0</code>	Erro RN1	Parcial (só 0)
CE2 — válida padrão	<code>0 < valor ≤ 100</code>	"padrão"	Sim
CE3 — válida cara	<code>valor > 100</code>	"cara"	Sim
CE4 — não-numérica	<code>undefined, null, NaN, string</code>	<i>indefinido na spec</i>	Não

A classe CE1 é testada apenas com o valor `0`; um valor negativo (`-10`) nunca é exercitado, embora o nome do teste afirme cobrir "zero ou negativo". CE4 não é considerada em nenhum teste.

4.2 Análise de valor-limite (boundary) — fronteiras 0 e 100

Valor	Fronteira	Esperado	Testado?
<code>-0.01 / -10</code>	Abaixo de 0	Erro RN1	Não
<code>0</code>	Limite inferior inválido	Erro RN1	Sim
<code>0.01</code>	Menor valor válido	"padrão"	Não
<code>99.99</code>	Logo abaixo de 100	"padrão"	Sim
<code>100</code>	Limite exato (exclusivo)	"padrão"	Sim
<code>100.01</code>	Menor valor "cara"	"cara"	Não

As fronteiras-chave `100` e `0` são testadas, o que é positivo. Faltam os pontos imediatamente *acima* de cada fronteira (`0.01` e `100.01`), que confirmam o lado oposto da decisão.

4.3 Tabela de decisão — método `pagar`

Condição	R1	R2	R3
<code>valor ≤ 0</code>	V	F	F
<code>valor > 100</code>	—	F	V
Ação	Lança erro RN1	Registra "padrão"	Registra "cara"
Caso de teste correspondente	Teste 2 (valor 0)	Testes 4 e 5	Teste 3

4.4 MC/DC — cobertura de condição/decisão modificada

O código possui três decisões, cada uma com **uma única condição** (sem operadores `&&` / `||`). Para predicados de condição única, MC/DC equivale à cobertura de ramos: basta exercitar cada decisão como Verdadeira e Falsa.

Decisão	Verdadeira (V)	Falsa (F)	MC/DC
D1 <code>valor > 100</code>	Teste 3 (150)	Testes 4/5 (99.99 / 100)	Atendida
D2 <code>valor ≤ 0</code>	Teste 2 (0)	Demais testes (> 0)	Atendida
D3 <code>length === 0</code>	Teste 7 (vazio)	Testes 1/8/... (não-vazio)	Atendida

Conclusão MC/DC: estruturalmente 100% — todas as decisões são exercitadas em ambos os sentidos. A limitação *não* é estrutural, e sim semântica: os próprios predicados não protegem contra entradas não-numéricas (ver Seção 6), portanto MC/DC pleno não impede os defeitos.

5. Suíte implementada — 28 testes por técnica (AAA)

Resultado da execução: **23 passing · 5 pending**. Cada teste é estruturado em Arrange / Action / Assert e comentado explicando o que cobre.

Seção / técnica	Testes	Cobre	Status
Caminho feliz & contrato	2	Registro de dados e propriedades de retorno	2 PASS
Partição de Equivalência	4	CE1 (zero e negativo), CE2 "padrão", CE3 "cara"	4 PASS
Análise de Valor-Limite	5	-0.01, 0.01, 99.99, 100, 100.01	5 PASS
Tabela de Decisão (data-driven)	3	Regras R1 (erro), R2 (padrão), R3 (cara)	3 PASS
MC/DC	6	D1, D2, D3 — cada decisão em V e F	6 PASS
Múltiplos pagamentos (RN4/RN5)	3	Sequência, último registro e lista vazia	3 PASS
Defeitos conhecidos (it.skip)	5	BUG-D1 a D5 (asserção = comportamento correto)	5 PENDING

Observação de QA: há sobreposição intencional entre técnicas (ex.: o valor 100 aparece em Valor-Limite e em MC/DC). Isso é proposital — cada teste demonstra a técnica de forma independente e auto-contida, em vez de reaproveitar um único caso para múltiplos propósitos.

Os 5 testes pendentes (regressão pronta)

ID	Teste (it.skip)	Técnica
BUG-D1	NaN deveria ser rejeitado	Partição CE4 (não-numérica)
BUG-D2	undefined deveria ser rejeitado	Partição CE4 (não-numérica)
BUG-D3	string "200" deveria ser rejeitada sem coerção	Partição CE4 · teste de tipo
BUG-D4	codigoBarras/empresa obrigatórios	Validação de campos obrigatórios
BUG-D5	retorno não deve vaziar a referência interna	Imutabilidade / encapsulamento

6. Defeitos & lacunas detectados (comprovados)

Apesar dos 100% de cobertura, os casos abaixo foram **executados contra o código real** e revelam comportamentos não cobertos pela spec. Na v2 cada um já está **codificado como teste `it.skip`** (BUG-D1 a D5) com a asserção do comportamento correto, pronto para reativar após a correção:

#	Entrada	Resultado observado	Esperado / risco	Sev.
D-1	<code>pagar("x", "E", NaN)</code>	Registra pagamento (<code>NaN>100</code> é F, <code>NaN≤0</code> é F)	Deveria rejeitar — pagamento com valor inválido aceito	Alta
D-2	<code>pagar("x", "E", undefined)</code>	Registra; objeto sem campo <code>valor</code>	Deveria rejeitar — registro corrompido	Alta
D-3	<code>pagar("x", "E", "200")</code>	Aceita string; categoriza "cara" por coerção	Sem checagem de tipo — comportamento frágil	Média
D-4	<code>pagar("", null, 50)</code>	Aceita código de barras vazio e empresa nula	Campos obrigatórios não validados	Média
D-5	Mutar objeto retornado por <code>consultar...</code>	Altera o estado interno <code>#pagamentos</code>	Vazamento de encapsulamento — retornar cópia	Média

Observação de design — ordem das operações

Em `pagar`, a categoria é calculada e o objeto é mont